Titanic: Machine Learning from Disaster

System Analysis Project

Julián Carvajal Garnica
20242020024

Andrés Mauricio Cepeda Villanueva
20242020010

Jhonatan David Moreno Barragán
20201020094

Andrés Camilo Ramos Rojas
20242020005

School of Engineering, Universidad Distrital Francisco José de Caldas
System Analysis Course
Teacher: Carlos Andrés Sierra
Bogotá D.C.
2025

Part 3 of the System Analysis Project

1. Review and Refine System Architecture

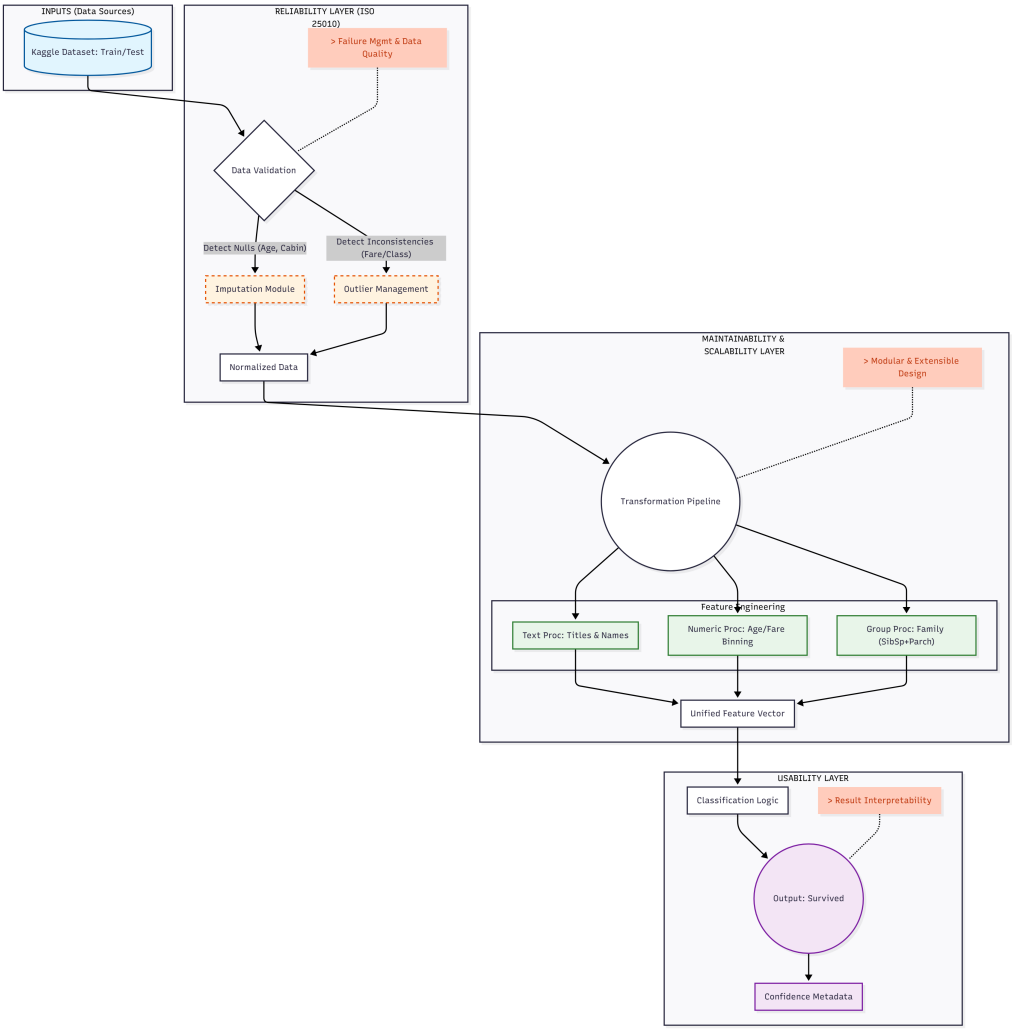

Figure 1: Robust System Architecture Diagram showing the Reliability, Maintainability, and Usability layers.

Architectural Evolution: From Logical View to Robust Design

The architecture diagram has been significantly evolved from the initial logical representation presented in Workshop 1. While the first iteration focused on identifying system boundaries (Input-Process-Output), this refined version incorporates specific engineering modules to address the **Sensitivity**, **Chaos**, and **Constraints** identified in the dataset analysis.

The key structural changes include:

- **Integration of a Reliability Layer (ISO 25010):** In Workshop 1, missing values (e.g., Age, Cabin) were identified as a source of system instability. The new diagram explicitly adds an *Imputation Module* and *Outlier Management* block *before* any processing occurs. This ensures that the core system is shielded from "dirty" data, directly addressing requirement **R1 (Handling Missing Data)**.
- **Modularization of the Processing Layer:** The original "Process" block has been decoupled into three independent feature engineering modules:
  - *Textual Processing* (for Names/Titles).
  - *Numeric Processing* (for Age/Fare binning).
  - *Group Processing* (for Family creation).

  This separation satisfies requirement **R5 (Maintainability)**, allowing individual components to be updated or debugged without disrupting the entire pipeline.
- **Enhanced Usability in Output:** Instead of a simple binary output, the architecture now includes *Confidence Metadata*. This addresses the user-centric requirement **U1 (Interpretability)**, acknowledging that users need to understand the certainty of a prediction, not just the result.

This layered approach transforms the system from a theoretical data model into a deployable, fault-tolerant software architecture.

1.1 Refinement of Robust Design Principles

The original architecture established a modular flow. We now refine this with an explicit focus on robustness:

- **Modularity and Maintainability (CMMI):** We are taking modularity a step further. Each component (Ingestion, Preprocessing, Modeling, Evaluation) [cite: 96-100] will be designed as a containerized microservice (e.g., using Docker). This not only reinforces modularity but also aligns with **CMMI** principles by allowing independent configuration management and facilitating maintainability, as one component can be updated without destabilizing the entire system.

- **Scalability:** The scalability NFR will be addressed through the orchestration of these containers (e.g., with Kubernetes). Specifically, the *Modeling Layer* and *Deployment Layer* will be able to scale horizontally. If the demand for Titanic predictions increases, we can dynamically launch more instances of the *Deployment* service.

- **Fault-Tolerance:** This is a critical refinement.

  - **Data Ingestion:** A message queue (e.g., RabbitMQ or Kafka) will be implemented between the ingestion and preprocessing layers. If the *Preprocessing Layer* fails, the raw Kaggle data is not lost; it waits in the queue, ensuring **reliability**.
  - **Deployment:** A load balancer will be used to distribute prediction requests among multiple model replicas. If one replica fails, traffic is automatically rerouted to healthy ones.
  - **Feedback Loop:** The *Feedback Layer* will monitor model health. If the error rate (an indicator of chaos) exceeds a threshold, the system can automatically roll back to a previously stable model version, ensuring **homeostasis**.

1.2 Components and their Support for Quality (ISO 9000)

Following the process management principle of **ISO 9000**, we define how each component ensures system quality and reliability:

- **Data Ingestion Layer:**

  - *Quality:* Implements data validation schemas (e.g., Pydantic) to ensure incoming data meets the expected format (data types, ranges).
  - *Reliability:* Uses retries and the message queue (see above) to handle network failures or Kaggle service downtime.

- **Preprocessing Layer:**

  - *Quality:* Versioning of preprocessing artifacts (e.g., Scikit-learn scalers or encoders) is implemented. Each trained model will be linked to the exact version of the preprocessing pipeline it used, ensuring **transparency** and reproducibility.

- **Modeling & Evaluation Layer:**

  - *Quality:* We apply a *Six Sigma* approach to "defect reduction" (prediction errors). The *Evaluation Layer* will not only measure accuracy but also use

SHAP and LIME to analyze the *root cause* of errors and bias, aligning with the "Analyze" phase of DMAIC.

- **Deployment & Feedback Layer:**
  - *Reliability:* Implements "Canary" deployments. A new model version is first released to a small percentage of users. The *Feedback Layer* monitors its real-time performance. If stable, it is rolled out to everyone; if it fails, it is rolled back without affecting the majority of users.

2. Quality and Risk Analysis

This section evaluates the main quality considerations and potential risks in the Titanic dataset system, proposing mitigation strategies and monitoring mechanisms that ensure reliability, stability, and robustness across all components of the architecture.

2.1 Quality Foundations

The design of the system follows established quality frameworks that guide process consistency, continuous improvement, and reliable operation:

- **ISO 9000 (Quality Management):** Provides process-oriented guidelines to ensure that each subsystem (ingestion, preprocessing, modeling, evaluation) follows consistent procedures and traceable workflows.
- **CMMI (Capability Maturity Model Integration):** Supports structured process control and progressive refinement of the system, ensuring that components evolve in a measurable and disciplined manner.
- **Six Sigma:** Encourages statistical stability and minimizes variation, particularly important for controlling sensitivity in the model's predictions and reducing noise introduced during preprocessing.

These frameworks ensure that quality is not treated as a static property but as an ongoing, iterative process embedded in the architecture.

2.2 Risk and Failure Analysis

The following table identifies key risks in the system, the proposed mitigation strategies, and the monitoring mechanisms that maintain system stability:

Risk / Failure Point	Description	Mitigation Strategy	Monitoring Mechanism
Data Quality Issues	Missing values (e.g., Age, Cabin), inconsistent formats, or corrupted entries may propagate errors into the model.	Implement validation scripts, schemas, and redundancy in data ingestion. Store all datasets in version-controlled repositories.	Automated validation logs, distribution checks, and alerts for anomalies during ingestion.
Model Sensitivity or Overfitting	The model may become unstable under slight input variations, reducing generalization.	Use regularization, cross-validation, ensemble models, and controlled perturbation tests to assess robustness.	Periodic sensitivity analysis, accuracy drift monitoring, and comparison against baseline validation metrics.
Pipeline or Deployment Failure	Interruptions in preprocessing, model loading, or inference services may halt the system.	Design modular pipeline components with retry logic, containerization, and fallback mechanisms for service continuity.	Continuous integration logs, automated health checks, and uptime dashboards.

2.3 Systemic Integration of Quality and Risk Control

Quality control and risk mitigation operate as interconnected feedback loops in the system. Each identified risk corresponds to a monitoring routine that continuously evaluates system behavior. When deviations appear—such as performance drops, unexpected data patterns, or service downtime—the system can take corrective measures to restore stability. This ensures the architecture maintains operational equilibrium, aligning with the principles of homeostasis and continuous improvement.

3. Project Management Plan

The Project Management Plan defines how the team organizes the development process, distributes responsibilities, structures the workflow, and establishes milestones that guide the construction of the Titanic predictive system. The plan is aligned with the findings of Workshops 1 and 2, ensuring that design decisions, risks, and quality considerations are properly managed throughout the project.

3.1 Team Roles and Responsibilities

To maintain clarity in development and ensure that all components of the system are addressed, the team assigns the following roles:

- **Project Manager (Andrés Camilo Ramos)**: Oversees task coordination, ensures that milestones are met, and manages communication among team members.
- **System Analyst (Julián Carvajal)**: Verifies alignment with system requirements, maintains consistency with Workshops 1 and 2, and ensures that the system architecture remains coherent.
- **Data Engineer (Jhonatan Moreno)**: Responsible for data ingestion, cleaning, and transformation, ensuring the integrity of the Kaggle dataset before it enters the system pipeline.
- **Machine Learning Developer (Andrés Cepeda)**: Trains, evaluates, and optimizes predictive models using the selected preprocessing pipeline and dataset.
- **QA Tester (All Members)**: Validates outputs, checks system behavior, documents errors, and ensures that sensitivity and stability recommendations are respected.

3.2 Milestones and Deliverables

The development timeline is structured into clear milestones that reflect the incremental construction of the system:

Milestone	**Description**	**Deadline**
System Refinement	Integration of design principles from Workshop 2 into the updated architecture.	Nov 16, 2025
Risk and Quality Analysis	Identification of failure points and mitigation strategies (ISO 9000 and sensitivity considerations).	Nov 19, 2025
Prototype Update	Implementation of the refined modules: preprocessing, feature engineering, and baseline model.	Nov 23, 2025
Final Documentation	Compilation of the complete Workshop 3 report, including diagrams and updated architecture.	Nov 28, 2025

3.3 Methodology and Tools

The project adopts an **Agile-inspired workflow** supported by a **Kanban board** to manage tasks in a transparent and iterative manner. This methodology allows the team to respond to findings from sensitivity and chaos analysis, adjusting tasks without disrupting overall progress.

Workflow Structure

Tasks are organized into the following Kanban columns:

- **To Do**: Planned tasks awaiting execution.
- **In Progress**: Tasks currently under development.
- **Completed**: Finalized and verified tasks.

Technical Workflow

The system development follows these sequential steps:

1. **Data Acquisition and Cleaning:** Importing `train.csv` and `test.csv`, handling missing values, validating formats.
2. **Feature Engineering:** Encoding categorical variables, extracting titles, grouping family variables (SibSp + Parch), and normalizing numerical attributes.
3. **Model Training and Evaluation:** Training baseline models (Decision Tree, Random Forest) and analyzing performance using accuracy and simple sensitivity checks.
4. **Prediction and Submission:** Generating the required `submission.csv` file following Kaggle constraints.

3.4 Timeline Overview

This timeline illustrates the sequential development of Workshop 3 deliverables, ensuring structured progress from architectural refinement to final documentation.

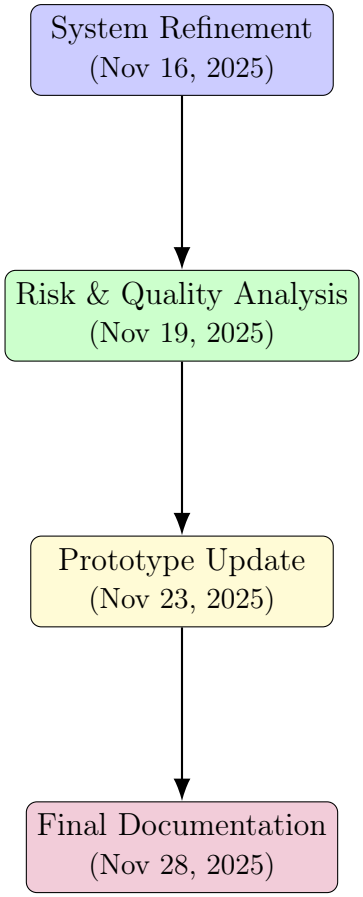

Figure 2: Project Workflow and Timeline Diagram

4. Incremental Improvements

Throughout Workshops 1 and 2, the system has undergone a progressive refinement process, transitioning from an initial descriptive analysis of the dataset to a structured, engineering-oriented design. The improvements implemented at each stage directly address the limitations, constraints, and sensitivity issues identified earlier, resulting in a more robust and coherent system architecture.

4.1 Improvements Derived from Workshop 1

Workshop 1 focused on understanding the Titanic dataset as a closed system composed exclusively of the variables provided by Kaggle. The main incremental contributions from that stage were:

- **Clear identification of system components**: Inputs (dataset attributes), processes (feature interactions), and outputs (Survived).
- **Recognition of structural constraints**: Missing values, categorical variables, and limited feature set requiring careful preprocessing.

- **Sensitivity and chaos insight**: Certain attributes (e.g., Sex, Pclass, Age) exhibit strong influence on the target, and small variations in these inputs may produce significantly different outputs.

These findings established the foundation for the requirements and design principles formalized in Workshop 2.

4.2 Improvements Derived from Workshop 2

Workshop 2 translated the analytical findings into engineering requirements and an initial architecture. The major improvements include:

- **Formalization of System Requirements**: Functional and non-functional requirements were derived directly from the systemic properties observed in Workshop 1 (e.g., need for consistent preprocessing, interpretability, and modularity).
- **Structured Architecture**: A high-level architecture was proposed with clearly defined layers (Ingestion, Preprocessing, Modeling, Evaluation, Output).
- **Sensitivity and Chaos Mitigation Strategies**: Using the system theory slides, we introduced buffering, normalization, redundancies, and monitoring mechanisms to stabilize the system against input perturbations.
- **Alignment with Systems Engineering Principles**: Modularity, maintainability, traceability, and controlled feedback loops were added to ensure stability and prevent error amplification.

These improvements transformed the system from a conceptual model into a structured, implementable pipeline.

4.3 Evolution Toward Robustness

Combining the previous insights, the system now incorporates:

- **A modular architecture** that supports independent modification of components.
- **Explicit handling of dataset weaknesses** such as missing values and inconsistent feature formats.
- **Sensitivity-aware preprocessing** to reduce amplified fluctuations in downstream components.
- **Feedback mechanisms** for monitoring model performance and detecting instability.

Overall, the design has matured from a static pipeline into a system that recognizes its own constraints, manages uncertainty, and adapts through controlled processes. This positions the project for a smoother transition into implementation and evaluation in the next workshop.